SIMATS Engineering

SIMATS Online Programming Lab — Python Programming (Mock Interview)

Course Name	Python Programming
Course Code	CSA0804
Name	Monish.L
Register Number	1972260035
Topic	CO3-AT3-MOCK INTERVIEW
Question	Q5 — List Comprehension with Nested Loops (10 Marks)

Q5. [10 Marks] Explain list comprehension syntax with nested loops. Trace the output: result = [(x, y) for x in range(3) for y in range(3) if x != y] print(result) How many elements does result contain?

Answer

List comprehension syntax with nested loops

A list comprehension is a concise way to build a new list by looping over one or more iterables and optionally filtering the results, all in a single expression enclosed in square brackets. The general syntax with a single loop is:

```
[expression for item in iterable if condition]
```

When nested loops are needed, additional 'for' clauses are simply chained one after another inside the same brackets, in the same left-to-right order they would appear if written as regular nested for loops:

```
[expression for x in iterable1 for y in iterable2 if condition]
```

This is exactly equivalent to writing an outer loop over x and, for each value of x, an inner loop over y, evaluating the expression and the optional condition on every combination — the first 'for' clause is the outermost loop, and each subsequent 'for' clause is nested one level deeper inside it. The optional 'if' clause, when present, acts as a filter: only combinations for which it is True are included in the resulting list.

Trace: result = [(x, y) for x in range(3) for y in range(3) if x != y]

This expression is equivalent to the nested loop below, which builds a tuple (x, y) for every combination of x and y from 0 to 2, but only keeps the tuple when x and y are not equal:

```
result = []
for x in range(3):
    for y in range(3):
        if x != y:
            result.append((x, y))
```

Since range(3) produces 0, 1, 2, there are 3 × 3 = 9 total (x, y) combinations. The condition x != y excludes the 3 combinations where x equals y — namely (0,0), (1,1), and (2,2) — leaving the following pairs in order: (0,1), (0,2), (1,0), (1,2), (2,0), (2,1).

```
result = [(0, 1), (0, 2), (1, 0), (1, 2), (2, 0), (2, 1)]
```

How many elements does result contain?

result contains **6 elements**. This follows from the total of 9 possible (x, y) pairs (3 choices for x times 3 choices for y) minus the 3 pairs excluded by the x != y condition, where x and y are equal, giving 9 – 3 = 6.

Python Program

```
"""
CO3-AT3 - Mock Interview
Q5: List Comprehension with Nested Loops [10 Marks]
------------------------------------------------------------
Traces a list comprehension with two nested for-clauses and
a filtering condition, and counts the resulting elements.
"""

result = [(x, y) for x in range(3) for y in range(3) if x != y]
print(result)
print("Number of elements:", len(result))
```

Sample Output

```
[(0, 1), (0, 2), (1, 0), (1, 2), (2, 0), (2, 1)]
Number of elements: 6
```

Conclusion

A list comprehension with nested loops reads left to right exactly like the equivalent nested for-loop statement, with the first 'for' clause as the outer loop and each following 'for' clause nested inside it, and an optional trailing 'if' clause filtering which combinations are kept. Tracing result = [(x, y) for x in range(3) for y in range(3) if x != y] shows that out of 9 possible (x, y) combinations from range(3), the 3 combinations where x equals y are filtered out, leaving exactly 6 tuples in the final list — demonstrating how list comprehensions combine iteration and filtering into a single, readable expression.